CSEC 101 // CYBERSECURITY LAB SERIES

01

LAB_01 > 01. Εισαγωγή στο Linux CLI

Εισαγωγή στο Linux CLI

Γραμμή Εντολών, Δικαιώματα Αρχείων & Βασικές Εντολές Δικτύου

Επίπεδο: Αρχάριος | bash | ls/grep/chmod | ss/ping | permissions

```
root@lab:~# ./start_lab_01.sh
```

Εργαστηριακό Υλικό Κυβερνοασφάλειας – Εκπαιδευτική Χρήση

VulnHub / TryHackMe, Ethical Hacking

Αναλυτικές Σημειώσεις — Εργαστήριο 01: Εισαγωγή στο Linux CLI

Το αρχείο αυτό είναι η **εμπλουτισμένη έκδοση** του `README.md`.

Τι είναι το CLI και γιατί το μαθαίνουμε

Το **CLI (Command Line Interface)** είναι τρόπος αλληλεπίδρασης με τον υπολογιστή μέσω κειμένου: γράφετε μια εντολή, το **κέλυφος (shell)** τη διαβάζει, την εκτελεί και επιστρέφει έξοδο. Το κέλυφος που χρησιμοποιείτε συνήθως είναι το **bash** (Bourne Again SHell).

Γιατί όμως CLI και όχι γραφικό περιβάλλον;

Πλεονέκτημα	Εξήγηση
Αυτοματοποίηση	Μια εντολή μπορεί να τρέξει σε 1000 servers μέσω script ή SSH
Ακρίβεια	Βλέπετε ακριβώς τι εκτελέστηκε — απαραίτητο σε περιστατικά ασφάλειας
Ταχύτητα	`grep` σε 10.000 αρχεία σε δευτερόλεπτα
Διαθεσιμότητα	Κάθε Linux server έχει shell, όχι όλοι γραφικό περιβάλλον
Ίχνη (audit)	Κάθε εντολή καταγράφεται στο ιστορικό (`~/.bash_history`)

Η δομή μιας εντολής

```
ls   -lh   /var/log   >  out.txt
|     |     |           |
|     |     |           └─ ανακατεύθυνση εξόδου
|     |     └─ όρισμα (σε τι ενεργεί)
|     └─ παράμετρος/flag (πώς να ενεργήσει)
└─ όνομα εντολής
```

Η γενική μορφή είναι: **`εντολή [παράμετροι] [ορίσματα]`**. Οι παράμετροι ξεκινούν με `-` (σύντομες) ή `--` (μακροσκελείς) και συχνά συνδυάζονται: `ls -l -h` = `ls -lh`.

Πώς να χρησιμοποιήσετε αυτές τις σημειώσεις: ανοίξτε ένα τερματικό και εκτελέστε κάθε εντολή καθώς τη διαβάζετε. Η γνώση του CLI δεν αποκτάται με ανάγνωση.

Εργαστηριακή Άσκηση 0: Εισαγωγή στη Γραμμή Εντολών (CLI) & Βασικές Εντολές Δικτύου

Στόχοι της Άσκησης

Επεξήγηση Μέρους Α — το σύστημα αρχείων ως δέντρο

Το Linux οργανώνει τα πάντα σε **ένα ενιαίο δέντρο** που ξεκινά από τη ρίζα `/`. Δεν υπάρχουν «δίσκοι» με γράμματα όπως `C:` — οι δίσκοι *προσαρτώνται* (mount) σε καταλόγους.

```
/               ρίζα (root directory)
├── bin/        βασικά εκτελέσιμα (ls, cp, bash)
├── etc/        αρχεία ρυθμίσεων (configuration)
├── home/       προσωπικοί κατάλογοι χρηστών (/home/kali)
├── var/        μεταβλητά δεδομένα (logs, databases, web root)
├── tmp/        προσωρινά αρχεία (σβήνονται στην επανεκκίνηση)
├── usr/        προγράμματα χρήστη (usr/bin, usr/share)
└── proc/       εικονικό σύστημα αρχείων του πυρήνα (μόνο στη μνήμη)
```

Απόλυτη vs σχετική διαδρομή — η βασική διάκριση όλης της ενότητας:

- **Απόλυτη** ξεκινά με `/` και ισχύει από παντού: `/home/kali/lab1_workspace`.
- **Σχετική** ξεκινά από τον *τρέχοντα* κατάλογο: `project/src` σημαίνει «μέσα στον κατάλογο που είμαι τώρα».

Ειδικά σύμβολα:

Σύμβολο	Σημαίνει
`.`	ο τρέχων κατάλογος
`..`	ο γονικός κατάλογος (ένα επίπεδο πάνω)
`~`	ο προσωπικός κατάλογος του χρήστη (`/home/kali`)
`-`	ο *προηγούμενος* κατάλογος (μόνο για `cd`)
`*`	μπαλαντέρ: «οτιδήποτε»

Με την ολοκλήρωση της παρούσας εργαστηριακής συνεδρίας, οι φοιτητές/σπουδαστές αναμένεται να:

1. Εξοικειωθούν με το περιβάλλον διεπαφής γραμμής εντολών (CLI) και τη βασική σύνταξη εντολών.
2. Κατανοήσουν σε βάθος την πλοήγηση μέσω απόλυτων και σχετικών διαδρομών (`cd` , `cd ..` , `cd ../..` , `cd -`).
3. Εφαρμόζουν προχωρημένες παραμέτρους και μοτίβα αναζήτησης με την εντολή απαρίθμησης `ls` (`-lh` , `-lt` , `-R` , `*`).
4. Εφαρμόζουν τεχνικές ανακατεύθυνσης εξόδου και διασωλήνωσης (pipes).

5. Διαχειρίζονται τα δικαιώματα προσπέλασης αρχείων μέσω συμβολικών και αριθμητικών παραμέτρων.
6. Ελέγχουν και αναλύουν βασικές ρυθμίσεις και παραμέτρους συνδεσιμότητας δικτύου (`ifconfig`, `ip`, `ping`, `ss`, `nslookup`).

Εισαγωγή στο CLI: Δομή Εργαστηρίου & Προετοιμασία Ιεραρχίας

Επεξήγηση 1.1 — `pwd`, `mkdir -p` και γιατί η δομή έχει σημασία

- `pwd` (*print working directory*) τυπώνει την **απόλυτη** διαδρομή του τρέχοντος καταλόγου. Είναι το «πού βρίσκομαι;» του CLI και η πρώτη εντολή που εκτελείτε όταν χαθείτε.
- `mkdir` (*make directory*) δημιουργεί κατάλογο. Χωρίς παράμετρο αποτυγχάνει αν ο γονικός δεν υπάρχει:

```
mkdir a/b/c      # σφάλμα: ο a δεν υπάρχει
mkdir -p a/b/c   # δημιουργεί όλη τη διαδρομή
```

- `-p` (*parents*) έχει δύο ρόλους: δημιουργεί **όλους** τους ενδιάμεσους καταλόγους **και** δεν παραπονιέται αν ο κατάλογος υπάρχει ήδη (idempotent). Γι' αυτό χρησιμοποιείται παντού σε scripts.

Τι σημαίνει η άσκηση στην πράξη: το `lab1_workspace/project/{docs,src,build}` χρησιμοποιεί **brace expansion** — το bash επεκτείνει το `{docs,src,build}` σε τρία ξεχωριστά ορίσματα *πριν* εκτελεστεί η `mkdir`. Ισοδυναμεί με:

```
mkdir -p lab1_workspace/project/docs lab1_workspace/project/src lab1_workspace/project/build
```

Συνηθισμένο λάθος: να γράψετε `mkdir -p lab1_workspace/project/{docs, src}` **με κενό** μετά το κόμμα — τότε το bash δημιουργεί κατάλογο με όνομα `" src"` (κενό μέσα στο όνομα). Το κενό στα ονόματα αρχείων είναι νόμιμο αλλά δημιουργεί προβλήματα παντού αργότερα.

1. Εξακριβώστε τον αρχικό κατάλογο εργασίας σας:
   `bash pwd`

2. Δημιουργήστε την ακόλουθη δενδροειδή δομή για τις δοκιμές πλοήγησης:
   `bash mkdir -p lab1_workspace/project/{docs,src,build} mkdir -p lab1_workspace/backup`

3. Μεταβείτε στον κεντρικό φάκελο της άσκησης:
   `bash cd lab1_workspace pwd`

Εκτεταμένα Παραδείγματα Πλοήγησης με `cd` (Σχετικές & Απόλυτες

Διαδρομές)

Επεξήγηση 1.2 — πώς σκέφτεται το κέλυφος όταν γράφετε `cd ../..`

Το κέλυφος δεν «ξέρει» πού είστε, απλώς **συνθέτει** τη διαδρομή που δίνετε με την τρέχουσα. Αν βρίσκεστε στο `/home/user/lab1_workspace/project/src`:

Εντολή	Αποτέλεσμα	Αιτιολόγηση
`cd ..`	`/home/user/lab1_workspace/project`	αφαιρεί το τελευταίο τμήμα
`cd ../..`	`/home/user/lab1_workspace`	αφαιρεί δύο τμήματα
`cd .`	ίδιος κατάλογος	δεν αλλάζει τίποτα (χρήσιμο σε scripts)
`cd ~` ή `cd`	`/home/user`	ο προσωπικός κατάλογος
`cd -`	ο προηγούμενος	το κέλυφος θυμάται τον `$OLDPWD`
`cd /var/log`	`/var/log`	απόλυτη διαδρομή, αγνοεί πού είστε

Γιατί το `cd -` είναι χρήσιμο: όταν δουλεύετε εναλλάξ σε δύο καταλόγους (π.χ. `/var/www/html` και `/etc/apache2`), σας γλιτώνει από πληκτρολόγηση. Εμφανίζει μάλιστα τη διαδρομή στην οποία πήγατε, ώστε να ξέρετε πού βρίσκεστε.

Το `cd` χωρίς όρισμα πηγαίνει πάντα στο `$HOME`. Αν το `$HOME` δεν έχει οριστεί (σπάνιο, σε scripts), το αποτέλεσμα είναι απρόβλεπτο — γι' αυτό σε scripts γράφουμε `cd "$HOME" || exit 1`.

Ειδική περίπτωση: `cd ..` από τη ρίζα `/` σας αφήνει στο `/` (δεν υπάρχει «πάνω» από τη ρίζα).

Η εντολή `cd` (*Change Directory*) επιτρέπει τη μετακίνηση στο ιεραρχικό δέντρο καταλόγων.

1. **Μετάβαση σε βαθύτερο υποφάκελο (Σχετική διαδρομή):**
   `bash cd project/src pwd # Αποτέλεσμα: /home/user/lab1_workspace/project/src`

2. **Ανέβασμα κατά ένα επίπεδο (`..`):**
   `bash cd .. pwd # Βρίσκεστε πλέον στο: /home/user/lab1_workspace/project`

3. **Ανέβασμα κατά πολλαπλά επίπεδα ιεραρχίας ταυτόχρονα (`../..`):**
   `bash # Από το project/src/ μεταβείτε κατευθείαν δύο επίπεδα επάνω cd src cd ../.. pwd # Βρίσκεστε πίσω στον ριζικό φάκελο της άσκησης: lab1_workspace`

4. **Εναλλαγή στον αμέσως προηγούμενο κατάλογο (`cd -`):**
   Η παράμετρος `-` επιστρέφει αυτόματα στον κατάλογο όπου βρισκόσασταν πριν την τελευταία αλλαγή:
   `bash cd project/build cd - # Επιστροφή στο lab1_workspace cd - # Επιστροφή στο project/build`

5. **Μετάβαση στον προσωπικό κατάλογο χρήστη (`cd ~` ή `cd`):**
   `bash cd ~ pwd # Επιστροφή στον φάκελο εργασίας της άσκησης μέσω σχετικής διαδρομής: cd lab1_workspace`

6. **Μετάβαση σε αδελφό κατάλογο μέσω σχετικής διαδρομής:**
   Μεταβείτε από το `project/docs/` στο `backup/` χωρίς να περάσετε πρώτα από το root:
   `bash cd project/docs cd ../../backup pwd # Βρίσκεστε στο: lab1_workspace/backup cd ..`

Βήμα 1.3: Αναλυτική Χρήση και Παράμετροι της Εντολής `ls`

Επεξήγηση 1.3 — διαβάζοντας την έξοδο του `ls -l`

Το `ls -l` εμφανίζει **7 πεδία**. Παράδειγμα:

```
-rwxr-xr--  1  kali  developers  15240  Oct 24 10:15  script.sh
|└┘└┘└┘  |   |        |         |         |          |
| |  |  |   |        |         |         |          └ όνομα
| |  |  |   |        |         |         └ ημερομηνία τροποποίησης
| |  |  |   |        |         └ μέγεθος σε bytes
| |  |  |   |        └ ομάδα (group)
| |  |  |   └ ιδιοκτήτης (owner)
| |  |  └ αριθμός hard links
| |  └ δικαιώματα για others (λοιπούς)
| └ δικαιώματα για group
└ δικαιώματα για owner  →  rwx = read/write/execute
τύπος: - αρχείο, d κατάλογος, l σύνδεσμος, c/b συσκευή
```

Οι παράμετροι που χρειάζεστε πραγματικά:

Παράμετρος	Κάνει	Πότε τη χρησιμοποιείτε
`-l`	αναλυτική λίστα	πάντα όταν σας νοιάζουν δικαιώματα/ιδιοκτήτης
`-a`	δείχνει και κρυφά (`.` `..` `.env`)	ψάχνοντας ρυθμίσεις ή secrets
`-h`	μέγεθος σε K/M/G	πάντα μαζί με `-l`
`-t`	ταξινόμηση κατά χρόνο (νέα πρώτα)	«τι άλλαξε τελευταία;»
`-r`	αντιστροφή σειράς	`-ltr` : παλαιότερα πρώτα
`-R`	αναδρομικά σε υποκαταλόγους	χαρτογράφηση δομής
`-S`	ταξινόμηση κατά μέγεθος	βρίσκοντας μεγάλα αρχεία

Γιατί το `-ltr` είναι το «εργαλείο του διαχειριστή»: σε έναν κατάλογο με 500 logs, τα νεότερα είναι στο **τέλος** — ακριβώς δίπλα στον κέρσορα, χωρίς scrolling. Είναι ο κλασικός συνδυασμός για «τι μόλις έγραψε η εφαρμογή;».

Κρυφά αρχεία και ασφάλεια: αρχεία που ξεκινούν με `.` (π.χ. `.env`, `.gitconfig`, `.ssh/`) δεν εμφανίζονται με σκέτο `ls`. Σε έναν web server, το `ls -la` στο `/var/www/html` μπορεί να αποκαλύψει `.htaccess` ή backup αρχεία που δεν θα έπρεπε να είναι προσβάσιμα — γι' αυτό είναι βασικό βήμα enumeration.

Wildcards (globbing): το `*` επεκτείνεται **από το κέλυφος**, όχι από την `ls`. Δηλαδή το `ls *.c` γίνεται `ls main.c utils.c` πριν εκτελεστεί. Αν δεν ταιριάζει κανένα αρχείο, το bash περνά το `*.c` αυτούσιο και η `ls` παραπονιέται «No such file».

Η εντολή `ls` (*List*) εμφανίζει τα περιεχόμενα ενός καταλόγου. Οι διαφορετικές σημαίες (flags) τροποποιούν τις παρεχόμενες λεπτομέρειες.

1. **Δημιουργία δοκιμαστικών αρχείων για παρατήρηση:**
   `bash cd lab1_workspace/project/src touch main.c utils.c config.h .env_secret echo "Linux CLI Lab" > notes.txt`

2. **Απλή προβολή έναντι αναλυτικής λίστας (`ls` vs `ls -l`):**
   `bash ls ls -l`
   Παρατηρήστε τα 7 πεδία της αναλυτικής προβολής: τύπος αρχείου/δικαιώματα, αριθμός δεσμών, ιδιοκτήτης, ομάδα, μέγεθος σε bytes, χρονοσφραγίδα τροποποίησης, όνομα.

3. **Εμφάνιση κρυφών αρχείων (`ls -a` και `ls -la`):**
   Στο Linux, τα αρχεία των οποίων το όνομα ξεκινά με τελεία (`.`) είναι κρυφά:
   `bash ls -la`
   Εντοπίστε τα αρχεία `.`, `..` και το κρυφό `.env_secret`.

4. **Ανάγνωση μεγέθους σε αναγνώσιμη μορφή (`ls -lh`):**
   Η παράμετρος `-h` (*human-readable*) μετατρέπει τα bytes σε K, M, G:
   `bash ls -lh`

5. **Ταξινόμηση βάσει χρόνου τροποποίησης (`ls -lt` και `ls -ltr`):**
   * `ls -lt`: Ταξινόμηση από το πιο πρόσφατο στο παλαιότερο.
   * `ls -ltr`: Αντίστροφη ταξινόμηση (τα πιο πρόσφατα αρχεία εμφανίζονται στο τέλος της οθόνης):
   `bash ls -ltr`

6. **Αναδρομική προβολή υποκαταλόγων (`ls -R`):**
   Εμφανίζει όλα τα περιεχόμενα όλων των υποφακέλων σε δενδροειδή μορφή:
   `bash cd ../.. ls -R lab1_workspace`

7. **Φιλτράρισμα με χαρακτήρες μπαλαντέρ (Wildcards / Globbing):**
   * Εμφάνιση μόνο των αρχείων με κατάληξη `.c`:
   `bash ls lab1_workspace/project/src/*.c`
   * Εμφάνιση αρχείων των οποίων το όνομα ξεκινά από `u`:
   `bash ls lab1_workspace/project/src/u*`

> **Ερώτημα 1.1:** Ποια είναι η διαφορά μεταξύ της εντολής `cd ..` και της εντολής `cd .`;
> **Ερώτημα 1.2:** Ποιος συνδυασμός παραμέτρων της `ls` είναι προτιμότερος όταν αναζητάτε ποιο αρχείο τροποποιήθηκε τελευταίο σε έναν κατάλογο με εκατοντάδες αρχεία;

Μέρος Β: Φιλτράρισμα Δεδομένων και Διασωλήνωση (Pipelines)

Επεξήγηση Μέρους Β — οι τρεις ροές δεδομένων

Κάθε πρόγραμμα στο Linux έχει **τρεις προεπιλεγμένες ροές**:

Ροή	FD	Σημασία	Προεπιλογή
stdin	0	είσοδος	πληκτρολόγιο
stdout	1	κανονική έξοδος	οθόνη
stderr	2	μηνύματα σφάλματος	οθόνη

Η δύναμη του CLI προέρχεται από το ότι αυτές οι ροές **επαναδρομολογούνται**:

```
εντολή > αρχείο      # stdout στο αρχείο (αντικαθιστά)
εντολή >> αρχείο     # stdout στο αρχείο (προσθέτει στο τέλος)
εντολή 2> err.txt    # stderr σε αρχείο
εντολή 2>/dev/null   # πετάει τα σφάλματα (στο «πηγάδι» του πυρήνα)
εντολή > out 2>&1    # και τα δύο στο ίδιο αρχείο
εντολή < αρχείο      # stdin από αρχείο
εντολή1 | εντολή2    # stdout της 1ης γίνεται stdin της 2ης
```

Γιατί το `2>/dev/null` εμφανίζεται παντού στην ασφάλεια: όταν τρέχετε `find /` ως απλός χρήστης, παίρνετε εκατοντάδες «Permission denied» στο stderr. Κρύβοντας το fd 2 μένει μόνο το χρήσιμο stdout — και είναι πιο γρήγορο από το να φιλτράρετε με `grep -v`.

Βήμα 2.1: Χρήση `grep` και Ανακατεύθυνση

Επεξήγηση 2.1 — `grep` ως φίλτρο γραμμών

Το **`grep`** (*global regular expression print*) διαβάζει γραμμές και τυπώνει όσες **ταιριάζουν** με ένα μοτίβο.

```
grep "model name" /proc/cpuinfo       # ψάχνει μέσα σε αρχείο
cat file | grep -i error              # -i: χωρίς διάκριση πεζών/κεφαλαίων
grep -c "Failed" /var/log/auth.log    # -c: μετράει γραμμές
grep -v "INFO" app.log                # -v: αντίστροφα (όσες ΔΕΝ ταιριάζουν)
grep -rn "password" /var/www          # -r αναδρομικά, -n με αριθμό γραμμής
grep -E "error|warn" app.log          # -E: extended regex (ή |)
```

Τι είναι το `/proc`: εικονικό σύστημα αρχείων που δημιουργεί ο **πυρήνας** στη μνήμη. Δεν υπάρχει στον δίσκο, κάθε ανάγνωση παράγει δεδομένα τη στιγμή εκείνη. Το `/proc/cpuinfo` δίνει πληροφορίες CPU, το `/proc/meminfo` μνήμη, το `/proc/net/tcp` ενεργές συνδέσεις.

Γιατί ανακατεύθυνση και όχι απλή εκτύπωση: το `>` **δημιουργεί/κενώνει** το αρχείο. Προσοχή: `grep x file > file` καταστρέφει το `file` (το κενώνει πριν διαβαστεί). Σωστό: `grep x file > out.txt`.

Σχετικό με ασφάλεια: το `grep -rn "password"` είναι μια από τις πρώτες ενέργειες μετά την απόκτηση πρόσβασης σε μηχάνημα — ψάχνει credentials σε κώδικα και ρυθμίσεις (θα το δείτε ξανά στο DC-1 με το `wp-config.php`).

Αναζητήστε πληροφορίες για τον επεξεργαστή από το εικονικό σύστημα αρχείων `/proc`:

```
grep "model name" /proc/cpuinfo > cpu_spec.txt
head -n 2 cpu_spec.txt
```

Βήμα 2.2: Διασωλήνωση (Pipeline)

Επεξήγηση 2.2 — γιατί το `|` αλλάζει τα πάντα

Το **pipe** `|` συνδέει το stdout της αριστερής εντολής με το stdin της δεξιάς **ταυτόχρονα** (streaming): δεν περιμένει να τελειώσει η πρώτη, ούτε γράφει σε προσωρινό αρχείο.

```
ps aux | wc -l            # πόσες διεργασίες
ps aux | grep sshd        # μόνο η sshd
ps aux | grep sshd | grep -v grep   # χωρίς την ίδια τη grep
ls -l | sort -k5 -n | tail -3       # τα 3 μεγαλύτερα αρχεία
```

Γιατί χρειάζεται το `grep -v grep`: η ίδια η `grep sshd` εμφανίζεται στη λίστα διεργασιών και ταιριάζει με τον εαυτό της. Είναι το κλασικό «false positive» των pipelines.

Ανάλυση της απάντησης: το `ps aux | wc -l` μετρά **γραμμές**, και η πρώτη γραμμή είναι η **επικεφαλίδα** (`USER PID %CPU ...`). Άρα ο πραγματικός αριθμός διεργασιών είναι `αποτέλεσμα - 1`. Αυτή η λεπτομέρεια εξετάζεται συχνά.

Το pipeline ως φιλοσοφία: κάθε εντολή κάνει **ένα** πράγμα καλά και επικοινωνεί με κείμενο. Έτσι συνδυάζονται μικρά εργαλεία σε σύνθετες ροές — η ίδια λογική που θα δείτε αργότερα στο `grep` των logs ενός SIEM.

Καταμετρήστε το πλήθος των ενεργών διεργασιών του συστήματος συνδυάζοντας την εντολή `ps` με την εντολή καταμέτρησης `wc`:

```
ps aux | wc -l
```

> **Ερώτημα 2.1:** Ποιος είναι ο ρόλος του χαρακτήρα διασωλήνωσης `|` (pipe) στη ροή δεδομένων μεταξύ δύο εντολών;

Μέρος Γ: Βασικές Εντολές Ελέγχου Δικτύου

Επεξήγηση Μέρους Γ — το μοντέλο που χρειάζεστε

Για να καταλάβετε τις εντολές, κρατήστε τρία επίπεδα:

Επίπεδο	Τι υπάρχει	Παράδειγμα
L2 (Data Link)	MAC addresses, switches	`00:0c:29:65:4d:49`
L3 (Network)	IP addresses, routers	`192.168.35.131`
L4 (Transport)	θύρες, TCP/UDP	`22/tcp`, `80/tcp`

Θύρα (port) = «πόρτα» σε μια IP που ξεχωρίζει **υπηρεσίες**. Η ίδια IP μπορεί να τρέχει SSH (22), HTTP (80) και MySQL (3306) ταυτόχρονα, η θύρα καθορίζει σε ποια μιλάτε.

Εύρος θυρών: 0–1023 *well-known* (απαιτούν root), 1024–49151 *registered*, 49152–65535 *dynamic*. Γι' αυτό ένας μη προνομιούχος χρήστης δεν μπορεί να «ακούσει» στη θύρα 80.

Βήμα 3.1: Έλεγχος Διασυνδέσεων Δικτύου (`ifconfig` / `ip`)

Επεξήγηση 3.1 — διαβάζοντας το `ip addr`

Το `ifconfig` ανήκει στο πακέτο `net-tools` και έχει **καταργηθεί** σε σύγχρονες διανομές, ο αντικαταστάτης είναι το `ip` (πακέτο `iproute2`). Χρήσιμες αντιστοιχίες:

Παλιό	Νέο
`ifconfig`	`ip addr` ή `ip a`
`ifconfig eth0 up`	`ip link set eth0 up`
`route -n`	`ip route`
`netstat -tuln`	`ss -tuln`
`arp -a`	`ip neigh`

Τα πεδία που κοιτάτε:

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    link/ether 08:00:27:aa:bb:cc brd ff:ff:ff:ff:ff:ff
    inet 192.168.35.129/24 brd 192.168.35.255 scope global eth0
```

- `UP` → η διεπαφή είναι ενεργή.
- `link/ether` → η **MAC** διεύθυνση (L2).
- `inet ... /24` → η **IPv4** και το μέγεθος δικτύου (`/24` = 256 διευθύνσεις, 254 χρησιμοποιήσιμες).
- `inet6` → IPv6 (αν υπάρχει).

Η διεπαφή `lo` (loopback):

- Είναι **εσωτερική** του μηχανήματος, δεν βγαίνει ποτέ στο καλώδιο.
- Έχει πάντα `127.0.0.1/8` (και `::1` σε IPv6).
- Χρησιμεύει ώστε οι τοπικές υπηρεσίες να μιλούν μεταξύ τους με το ίδιο API που θα χρησιμοποιούσαν απομακρυσμένα.
- Αν μια υπηρεσία ακούει **μόνο** σε `127.0.0.1`, δεν είναι προσβάσιμη από άλλο μηχάνημα — σημαντική λεπτομέρεια σε enumeration.

Στο εργαστήριο: η IP της VM (π.χ. `192.168.35.129`) είναι αυτή που θα χρησιμοποιήσετε ως «επιτιθέμενη» διεύθυνση, και η IP του στόχου θα βρεθεί με `netdiscover` / `nmap`.

1. Εμφανίστε τις διαθέσιμες διεπαφές δικτύου και τις αντιστοιχισμένες διευθύνσεις IP:
   `bash ifconfig`
   (Σε περίπτωση που η εντολή `ifconfig` δεν είναι εγκατεστημένη στο σύστημα, εκτελέστε την εντολή `ip addr show`).

2. Εντοπίστε τη διεύθυνση IPv4 (`inet`) της κύριας διεπαφής (π.χ. `eth0`, `enp3s0` ή `wlan0`), καθώς και της τοπικής διεπαφής ανάδρασης (`lo`).

> **Ερώτημα 3.1:** Ποιος είναι ο ρόλος της διεπαφής `lo` (loopback) και ποια είναι η προεπιλεγμένη διεύθυνση IPv4 που της αντιστοιχεί;

Βήμα 3.2: Έλεγχος Συνδεσιμότητας με την Εντολή `ping`

Επεξήγηση 3.2 — τι λέει πραγματικά ένα `ping`

Το `ping` στέλνει **ICMP Echo Request** και περιμένει **Echo Reply**. Είναι το απλούστερο τεστ «υπάρχει και απαντά;».

Ανάγνωση της εξόδου:

```
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.043/0.055/0.062/0.008 ms
```

Μετρική	Σημασία
packet loss	% πακέτων που χάθηκαν — 0% σε υγιές τοπικό δίκτυο
rtt (round-trip time)	χρόνος μετάβασης-επιστροφής, σε ms
mdev	διακύμανση (jitter) — υψηλό σημαίνει ασταθές δίκτυο

Ερμηνεία σε τοπικό δίκτυο: `rtt < 1 ms` είναι φυσιολογικό. `> 10 ms` σε LAN υποδηλώνει πρόβλημα (φόρτος, κακή εικονική κάρτα, λάθος δίκτυο).

Γιατί το `ping` δεν είναι αξιόπιστο για «είναι ζωντανό;»: πολλοί hosts/τείχη **μπλοκάρουν** ICMP. Ένα «100% packet loss» δεν σημαίνει απαραίτητα νεκρό μηχάνημα — γι' αυτό στα CTF χρησιμοποιούμε **ARP** (`netdiscover`) ή TCP scans (`nmap -Pn`).

`-c N` : στέλνει N πακέτα και σταματά. Χωρίς αυτό, το ping τρέχει **επ' άπειρον** μέχρι `Ctrl+C` — μην το αφήνετε σε script χωρίς όριο.

1. Ελέγξτε τη λειτουργία της τοπικής στοίβας δικτύου:
   `bash ping -c 3 127.0.0.1`
2. Ελέγξτε τη συνδεσιμότητα με έναν εξωτερικό κόμβο (π.χ. διακομιστής DNS), αποστέλλοντας ακριβώς 4 πακέτα αίτησης ICMP:
   `bash ping -c 4 8.8.8.8`

> **Ερώτημα 3.2:** Καταγράψτε το ποσοστό απώλειας πακέτων (packet loss) και τη μέση τιμή χρόνου απόκρισης (average round-trip time - RTT) από την έξοδο του `ping` .

Βήμα 3.3: Ενεργές Συνδέσεις και Επίλυση Ονομάτων (`ss` , `nslookup` , `traceroute`)

Επεξήγηση 3.3 — θύρες που ακούν, ονόματα και διαδρομές

`ss -tuln` — ανάλυση των παραμέτρων:

Σημαία	Σημασία
`-t`	TCP
`-u`	UDP
`-l`	**listening** (θύρες που περιμένουν σύνδεση)
`-n`	αριθμητικά (χωρίς reverse DNS — γρηγορότερα)

Η έξοδος δείχνει `Local Address:Port` . Το `0.0.0.0:22` σημαίνει «ακούει σε **όλες** τις διεπαφές» (προσβάσιμο από δίκτυο), το `127.0.0.1:3306` σημαίνει «μόνο τοπικά» (MySQL όχι εκτεθειμένη). Αυτή η διάκριση είναι **κρίσιμη** σε audit.

`nslookup www.kernel.org` — ρωτά τον **DNS resolver**:

```
Server:  192.168.1.1        ← ποιος απάντησε
Address: 192.168.1.1#53
Name:    www.kernel.org
Address: 139.178.84.217     ← η εγγραφή A (IPv4)
```

Αν αποτύχει: δεν έχετε DNS server ρυθμισμένο (`/etc/resolv.conf`) ή δεν έχετε δίκτυο. Εναλλακτικά: `dig` , `host` , `getent hosts` .

`traceroute 8.8.8.8` — δείχνει τα **hops** (routers) μέχρι τον προορισμό. Λειτουργεί αυξάνοντας το **TTL**: κάθε router που λαμβάνει πακέτο με TTL=1 το πετά και απαντά «time exceeded», αποκαλύπτοντας τη διεύθυνσή του. Χρήσιμο για τον εντοπισμό του σημείου όπου διακόπτεται η σύνδεση, καθώς και για τη χαρτογράφηση του δικτύου.

Σημείωση: σε NAT (όπως στο VirtualBox) το traceroute συχνά δείχνει `* * *` γιατί οι ενδιάμεσοι δεν απαντούν — δεν είναι σφάλμα.

1. Ελέγξτε τις ανοικτές και ακροώμενες θύρες TCP/UDP του συστήματός σας:
   `bash ss -tuln # Εναλλακτική παραδοσιακή εντολή: netstat -tuln`
2. Εκτελέστε επίλυση ονόματος τομέα (DNS resolution) για έναν ιστότοπο:
   `bash nslookup www.kernel.org`
3. Ανιχνεύστε τα ενδιάμεσα δρομολογητικά άλματα (hops) μέχρι τον προορισμό:
   `bash traceroute 8.8.8.8`

Μέρος Δ: Διαχείριση Δικαιωμάτων Αρχείων (`chmod`)

Επεξήγηση Μέρους Δ — το μοντέλο δικαιωμάτων του Unix

Κάθε αρχείο έχει **ιδιοκτήτη (owner)**, **ομάδα (group)** και **τρεις κατηγορίες χρηστών** με τρία δικαιώματα:

Γράμμα	Δικαίωμα	Σε αρχείο	Σε κατάλογο
`r` (4)	read	ανάγνωση περιεχομένου	λίστα ονομάτων (`ls`)
`w` (2)	write	τροποποίηση	δημιουργία/διαγραφή αρχείων μέσα
`x` (1)	execute	εκτέλεση	**είσοδος** στον κατάλογο (`cd`)

Τα ψηφία προστίθενται: `rwx = 7` , `rw- = 6` , `r-x = 5` , `r-- = 4` .

Έτσι προκύπτει το `755` :

```
7 5 5
| | └─ others: r-x  (διαβάζουν και εκτελούν, δεν γράφουν)
| └─── group : r-x
└───── owner : rwx  (πλήρης έλεγχος)
```

Γιατί ΔΕΝ χρησιμοποιούμε `chmod 777`: δίνει **εγγραφή σε όλους**. Οποιοσδήποτε χρήστης μπορεί να αντικαταστήσει το αρχείο με κακόβουλο περιεχόμενο — κλασικό privilege escalation. Στο Raven 2 θα δείτε ακριβώς αυτό: `chmod 777 LinEnum.sh` είναι κακή πρακτική.

SUID / SGID / sticky bit:

Bit	Σε αρχείο	Σε κατάλογο
SUID (`4xxx`)	εκτελείται με δικαιώματα **ιδιοκτήτη**	—
SGID (`2xxx`)	εκτελείται με δικαιώματα **ομάδας**	νέα αρχεία παίρνουν την ομάδα του καταλόγου
sticky (`1xxx`)	—	μόνο ο ιδιοκτήτης σβήνει τα αρχεία του (`/tmp`)

Το SUID εμφανίζεται ως `s` στη θέση του `x`: `-rwsr-xr-x`. Είναι το βασικό εργαλείο του privilege escalation (DC-1: `/usr/bin/find` με SUID → root).

Οι αριθμητικές μορφές που θα συναντήσετε:

Τιμή	Σημασία	Χρήση
`600`	`rw-------`	ιδιωτικά αρχεία (κλειδιά SSH)
`644`	`rw-r--r--`	συνηθισμένα αρχεία, web content
`700`	`rwx------`	προσωπικοί κατάλογοι/scripts
`755`	`rwxr-xr-x`	εκτελέσιμα, κατάλογοι
`777`	`rwxrwxrwx`	**να αποφεύγεται**
`4755`	SUID + 755	εκτελέσιμο με δικαιώματα root

Βήμα 4.1: Τροποποίηση Δικαιωμάτων Εκτέλεσης

Επεξήγηση 4.1 — γιατί το `./script.sh` χρειάζεται το `+x`

Όταν γράφετε `script.sh`, το κέλυφος ψάχνει το πρόγραμμα `script.sh` στο `$PATH` (καταλόγους όπως `/usr/bin`). Ο **τρέχων κατάλογος δεν είναι στο `$PATH`** (για λόγους ασφαλείας — αλλιώς ένα κακόβουλο `ls` στον κατάλογο λήψεων θα εκτελούνταν). Γι' αυτό γράφουμε `./script.sh`: το `./` σημαίνει «εδώ, σε αυτόν τον κατάλογο».

Χωρίς το `x`, το σύστημα αρνείται: `Permission denied`, ακόμη κι αν το αρχείο ανήκει σε εσάς. Το δικαίωμα `x` είναι **άδεια εκτέλεσης**, όχι «σωστό περιεχόμενο».

Τι κάνει το `chmod +x` έναντι του `chmod 755`: το `+x` **προσθέτει** το δικαίωμα εκτέλεσης σε όλους, διατηρώντας τα υπόλοιπα ως έχουν (`644` → `755`). Το `755` **θέτει** την κατάσταση απόλυτα — αν το αρχείο ήταν `600`, γίνεται `755` και αποκτά ανάγνωση για όλους.

Η πρώτη γραμμή `#!/bin/bash` (shebang): λέει στον πυρήνα ποιος διερμηνέας θα εκτελέσει το script. Χωρίς αυτήν, το σύστημα χρησιμοποιεί το τρέχον shell (`sh`), που μπορεί να μην υποστηρίζει τη σύνταξη bash.

Έλεγχος πριν και μετά:

```
ls -l script.sh      # -rw-r--r--  (644)
chmod +x script.sh
ls -l script.sh      # -rwxr-xr-x  (755)
```

1. Δημιουργήστε ένα απλό αρχείο σεναρίου εντολών:
   `bash echo 'echo "Σενάριο Εργαστηρίου Εκτελέστηκε Επιτυχώς"' > script.sh`
2. Ελέγξτε τα αρχικά δικαιώματα προσπέλασης:
   `bash ls -l script.sh`
3. Προσθέστε δικαίωμα εκτέλεσης (`+x`) και εκτελέστε το:
   `bash chmod +x script.sh ./script.sh`
4. Ρυθμίστε τα δικαιώματα σε αριθμητική μορφή `755`:
   `bash chmod 755 script.sh`

> **Ερώτημα 4.1:** Αναλύστε τι δικαιώματα παρέχει η τιμή `755` για τον ιδιοκτήτη (owner), την ομάδα (group) και τους λοιπούς χρήστες (others).

Παράρτημα Α — Απαντήσεις στα ερωτήματα της άσκησης

Ερώτημα 1.1 — Διαφορά `cd ..` και `cd .`

Το `cd ..` μεταβαίνει στον **γονικό** κατάλογο (ένα επίπεδο πάνω), δηλαδή αλλάζει τον τρέχοντα κατάλογο εργασίας. Το `cd .` μεταβαίνει στον **ίδιο** κατάλογο (`.` = τρέχων), δηλαδή πρακτικά δεν αλλάζει τίποτα — χρησιμοποιείται σε scripts για επιβεβαίωση ύπαρξης/προσβασιμότητας καταλόγου, όχι για πλοήγηση.

Ερώτημα 1.2 — Ποιος συνδυασμός `ls` βρίσκει το τελευταία τροποποιημένο αρχείο

`ls -lt` (ή `ls -ltr`). Το `-t` ταξινομεί κατά **χρόνο τροποποίησης**, με το νεότερο πρώτο, το `-r` αντιστρέφει ώστε το νεότερο να είναι στο τέλος της οθόνης, δίπλα στον κέρσορα σε μεγάλες λίστες. Προσθέτοντας `-h` (`ls -ltrh`) τα μεγέθη γίνονται αναγνώσιμα.

Ερώτημα 2.1 — Ρόλος του `|` (pipe)

Το pipe συνδέει την **τυπική έξοδο (stdout)** της αριστερής εντολής με την **τυπική είσοδο (stdin)** της δεξιάς, χωρίς ενδιάμεσο αρχείο. Τα δεδομένα ρέουν **ζωντανά** (streaming), επιτρέποντας σύνθεση μικρών εργαλείων σε μία ροή επεξεργασίας (π.χ. `ps aux | grep sshd | wc -l`).

Ερώτημα 3.1 — Ρόλος του `lo` και προεπιλεγμένη διεύθυνση

Η `lo` (*loopback*) είναι εσωτερική εικονική διεπαφή που επιτρέπει σε διεργασίες του **ίδιου** μηχανήματος να επικοινωνούν μεταξύ τους μέσω της στοίβας TCP/IP, χωρίς φυσικό δίκτυο. Προεπιλεγμένη διεύθυνση IPv4: **`127.0.0.1/8`** (και `::1` για IPv6). Χρησιμεύει σε έλεγχο υπηρεσιών και σε απομόνωση: μια υπηρεσία που ακούει μόνο σε `127.0.0.1` δεν είναι προσβάσιμη από το δίκτυο.

Ερώτημα 3.2 — Packet loss και μέση RTT

Διαβάζονται από τις δύο τελευταίες γραμμές της εξόδου:

```
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.043/0.055/0.062/0.008 ms
```

Σε τοπικό δίκτυο αναμένεται **0% loss** και **μέση RTT < 1 ms**. Υψηλό loss (> 0%) ή RTT δεκάδων ms σε LAN υποδηλώνει πρόβλημα δικτύου ή υπερφόρτωση της VM.

Ερώτημα 4.1 — Ανάλυση του `755`

Κατηγορία	Δυαδικό	Δικαιώματα	Πρακτικά
owner	111	`rwx`	διαβάζει, γράφει, εκτελεί
group	101	`r-x`	διαβάζει, εκτελεί — **όχι** εγγραφή
others	101	`r-x`	διαβάζει, εκτελεί — **όχι** εγγραφή

Είναι η τυπική τιμή για εκτελέσιμα και καταλόγους: ο ιδιοκτήτης συντηρεί, οι υπόλοιποι μόνο εκτελούν.

Παράρτημα Β — Γλωσσάρι

Όρος	Σημασία
Shell	Διερμηνέας εντολών (bash, zsh, sh)
stdin / stdout / stderr	Ροές 0 / 1 / 2
Pipe (`|`)	Σύνδεση stdout → stdin
Redirect (`>` , `>>` , `2>`)	Επαναδρομολόγηση ροής σε αρχείο
Wildcard / glob	Επέκταση μοτίβου από το κέλυφος (`*` , `?` , `[abc]`)
FHS	Filesystem Hierarchy Standard (η δομή `/bin` , `/etc` , `/var`)
Mount	Προσάρτηση συστήματος αρχείων σε κατάλογο
Loopback	Εσωτερική διεπαφή 127.0.0.1
TTL	Time To Live — όριο hops σε πακέτο
DNS resolver	Ο server που μετατρέπει όνομα → IP
SUID	Εκτέλεση με δικαιώματα ιδιοκτήτη
Umask	Προεπιλεγμένα δικαιώματα νέων αρχείων (συνήθως 022 → 644/755)

Παράρτημα Γ — Λίστα ελέγχου δεξιοτήτων

- [ ] Ξέρω πού βρίσκομαι (`pwd`) και πώς πάω σε απόλυτη/σχετική διαδρομή
- [ ] Διαβάζω μια γραμμή `ls -l` και εξηγώ τα 7 πεδία
- [ ] Χρησιμοποιώ `grep` με `-i` , `-v` , `-c` , `-r` και συνδυάζω με pipes
- [ ] Εξηγώ τη διαφορά `>` και `>>` και γιατί το `2>/dev/null` κρύβει σφάλματα
- [ ] Βρίσκω την IP μου, κάνω ping και ερμηνεύω loss/RTT
- [ ] Διακρίνω «ακούει σε 0.0.0.0» από «ακούει σε 127.0.0.1»
- [ ] Υπολογίζω δικαιώματα από `rwx` σε αριθμητικό και αντίστροφα
- [ ] Εξηγώ γιατί το `777` είναι επικίνδυνο και τι σημαίνει SUID